

Funciones que se trabajaron:

isGoalState , getStepCost , getSuccessors

El problema utiliza un estado compuesto con el formato de position, hasModule. La posición en si no es suficiente ya que el robot puede encontrarse con la misma celda antes o después de recoger el módulo y esto tiene costo y condiciones de meta diferentes. Se completa cuando el robot esté dentro del control C y los hasModules es verdadero. Para generar los sucesores se tomarán en cuenta cuatro movimientos cardinales y se descartan los que llevan a un muro. Cuando el robot entra a la posición M el siguiente estado cambia a hasModule a verdadero. El costo de cada movimiento se da a partir del terreno de la posición de destino, si el robot no lleva el módulo se usa el costo normal pero después de recogerlo cuestan el doble. El movimiento usado para entrar a M conserva el costo normal porque el costo se calcula el valor de hasModule anterior a la transición.

Prompt usados:

Justificación y registro de interacción con IA:

Se hizo uso de la herramienta ChatGPT como acompañamiento para identificar dónde trabajar , comprender el estado compuesto y verificar el resultado.

Prompt principal:

“Quiero que me guíes y me expliques paso a paso para poder entender bien el y hacer el trabajo .”

Respuesta:

ChatGPT explicó que el estado debía separarse en position y hasModule que la meta exigía estar en self.controlPosition y tener el módulo que el costo debía calcularse con el estado anterior a la transición y que getSuccessors debía generar los cuatro movimientos cardinales, descartar muros y conservar el formato (nextState, direction, stepCost).

Cambios aceptados, rechazados o adaptados:

Acepté las explicaciones sobre el estado compuesto, la condición de meta, el cálculo del costo y la generación de sucesores. Rechacé que la IA implementara directamente toda la solución y pedí que me guiara paso a paso. Escribí personalmente el código, corregí los espacios detectados por git diff --check y verifiqué la implementación ejecutándola en los diez mapas de layouts module.

Verificación:

Primero verifiqué la sintaxis con:

```
python3 -m py_compile algorithms/problems.py
```

Después ejecuté ModuleRepairProblem con uniformCostSearch sobre los diez mapas de layouts module. Todas las ejecuciones finalizaron con el mensaje “¡Misión completada!”. Como prueba final, moduleVault encontró una solución de costo 80 y expandió 245 nodos. También ejecuté git diff --check para comprobar que el archivo no tuviera espacios sobrantes ni errores de formato. Finalmente revisé git status para confirmar que el único archivo modificado fuera algorithms problems.py.

Aprendizaje:

Durante este ejercicio entendí por qué la posición del robot no era suficiente como estado y era necesario incluir `hasModule`. También entendí que el costo del movimiento para entrar a M debe calcularse antes de actualizar ese valor, para que el movimiento conserve el costo normal. La orientación de la IA fue principalmente una guía para organizar la implementación y verificarla. Después de realizar las pruebas, considero que puedo explicar el funcionamiento de las tres funciones.